

SETUP — Continuar el proyecto en otra Mac

Guía para retomar **Kiosk Portrait / TrueOmni Studio** en una segunda Mac usando GitHub. Pensada para leerse **justo después de clonar** el repo. Todo en español. Última revisión: 2026-06-25.

0. Modelo mental — qué viaja por dónde

El proyecto se reconstruye en tres capas, cada una con su propio mecanismo. Ninguna sola basta; las tres juntas dejan la otra Mac idéntica a esta.

Capa	Qué incluye	Mecanismo
Código + estado	<code>src/</code> , <code>clients/</code> , <code>designs/</code> , <code>.planning/</code> (incluye <code>STATE.md</code> , la memoria operativa entre sesiones)	<code>git</code> (clone/pull/push)
Secrets + entorno	<code>.env.local</code> (~20 claves) y los CLIs (Node, pnpm, gh, vercel, agent-browser)	<code>vercel env pull</code> + Homebrew
Claude Code	skills/plugins globales + memoria de Claude (<code>~/.claude/.../memory</code>)	marketplace + repo de dotfiles (§7)

Regla de oro: git es el árbitro. No trabajes en las dos Macs a la vez; haz `pull` al empezar y `push` al terminar (sección 8).

1. Prerrequisitos (instalar una vez en la Mac nueva)

```
# 1. Xcode Command Line Tools (compiladores, git)
xcode-select --install

# 2. Homebrew
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 3. nvm + Node 20 (el repo fija Node >=20 vía .nvmrc)
brew install nvm
mkdir -p ~/.nvm
# añade a ~/.zshrc lo que indique `brew info nvm`, luego abre una terminal nueva
nvm install 20 && nvm use 20

# 4. pnpm 9.0.0 vía corepack (el repo lo fija en package.json → "packageManager")
corepack enable
corepack prepare pnpm@9.0.0 --activate

# 5. CLIs del proyecto
brew install gh
npm install -g vercel agent-browser

# 6. Claude Code
npm install -g @anthropic-ai/claude-code # o el instalador oficial de claude.ai
```

Verifica versiones: `node -v` (v20+), `pnpm -v` (9.0.0), `gh --version`, `vercel --version`, `agent-browser --version` (0.21.x), `claude --version`.

Autenticaciones:

```
gh auth login # cuenta GitHub con acceso a rubatrejo/TrueOmni-Studio
vercel login # usar ruba.trejo@gmail.com (el email vinculado al proyecto Vercel)
```

2. Clonar el repo — EN LA MISMA RUTA

⚠ **Importante:** la memoria de Claude se indexa por la **ruta absoluta** del proyecto. Para que la memoria (\$7) encaje sin renombrar nada, clona en la ruta idéntica a esta Mac:

```
mkdir -p ~/Documents/"Claude Code"
cd ~/Documents/"Claude Code"
git clone https://github.com/rubatrejo/TrueOmni-Studio.git "Kiosk Portrait Old"
cd "Kiosk Portrait Old"
```

Ruta final esperada: `~/Documents/Claude Code/Kiosk Portrait Old` (que Claude indexa como `-Users-<tu-usuario>-Documents-Claude-Code-Kiosk-Portrait-Old`).

Si tu usuario de macOS NO es `rubenramirez`, el hash cambiará igualmente — en ese caso, tras el primer arranque

de Claude, renombra la carpeta de memoria al hash nuevo (ver nota en §7).

3. Secrets — `vercel env pull`

Los ~20 secrets de `.env.local` **no están en git** (y nunca deben estarlo). Ya viven en tu proyecto de Vercel, así que se reproducen exactos:

```
vercel link                # scope: rubatrejo-gmailcoms-projects · project: trueomni-studio
vercel env pull .env.local  # baja las env vars del proyecto a .env.local
```

`vercel env pull` baja por defecto el entorno **Development**. Si tras el smoke (§4) algo falla por una variable ausente, prueba: `vercel env pull .env.local --environment=production`

Comprueba que estén las claves esperadas (solo nombres; sin valores):

```
AUTH_GITHUB_ID · AUTH_GITHUB_SECRET · AUTH_SECRET · AUTH_TRUST_HOST
KV_URL · KV_REDIS_URL · KV_REST_API_URL · KV_REST_API_TOKEN · KV_REST_API_READ_ONLY_TOKEN
BLOB_READ_WRITE_TOKEN · STUDIO_GITHUB_OWNER · STUDIO_GITHUB_REPO · STUDIO_GITHUB_BRANCH
STUDIO_GITHUB_TOKEN · STUDIO_ADMIN_EMAILS · KIOSK_CLIENT
TAVUS_API_KEY · TAVUS_API_URL · TAVUS_PERSONA_ID · TAVUS_REPLICA_ID
```

```
grep -oE '^[A-Z0-9_]+= ' .env.local | sed 's/=/' | sort  # listar lo descargado
```

4. Dependencias + smoke test

```
pnpm install
pnpm kiosk:dev      # abre http://localhost:3000 → debe responder 200
pnpm typecheck      # limpio
pnpm lint           # limpio
```

Si el kiosk levanta y `typecheck` / `lint` pasan, el entorno del proyecto está listo.

5. Claude Code — skills, plugins y comandos

El primer arranque pide login:

```
cd ~/Documents/"Claude Code"/"Kiosk Portrait Old"
claude
```

Reinstala el marketplace y los plugins/skills globales (viven en `~/ .claude`, **fuera** del repo):

```
/plugin marketplace add anthropics/claude-plugins-official
/plugin install superpowers
/plugin install graphify
# (instala los demás que uses: figma, playground, agent-sdk-dev...)
```

- Los **comandos y agentes del proyecto** (`.claude/commands/` , `.claude/agents/`) ya vienen en el repo clonado — no hay que reinstalarlos.
- El CLI `agent-browser` ya quedó instalado en \$1 (toolchain único de QA visual).
- La **memoria de Claude** se trae aparte: ver \$7.

6. Verificación final (checklist "listo para trabajar")

- ☐ `node -v` ≥ 20 · `pnpm -v` = 9.0.0
- ☐ `pnpm kiosk:dev` responde 200 en `localhost:3000`
- ☐ `pnpm typecheck && pnpm lint` limpios
- ☐ `.env.local` con las ~20 claves (\$3)
- ☐ `gh auth status` y `vercel whoami` OK
- ☐ `claude` arranca; `/iniciar` lee `.planning/STATE.md`
- ☐ Memoria de Claude presente (\$7)

7. Memoria de Claude — repo de dotfiles (privado)

La memoria operativa entre sesiones ya viaja por git dentro de `.planning/STATE.md`. La memoria propia de Claude (`~/claude/.../memory/` , preferencias y feedback) **no** va al repo del proyecto; se sincroniza con un **repo privado de dotfiles**.

7a. Una vez, en ESTA Mac (crear el repo)

```
cd ~/.claude
git init -q claude-config 2>/dev/null || true
# Estructura: solo lo portable y NO sensible
# (memoria del proyecto + comandos/agentes/hooks globales + CLAUDE.md).
# Se EXCLUYE: caches, sessions, history, downloads, plugins y settings*.json
# (estos últimos pueden contener tokens — no subir sin revisar).
```

Contenido recomendado a versionar:

```
projects/-Users-rubenramirez-Documents-Claude-Code-Kiosk-Portrait-Old/memory/
commands/
agents/
hooks/
CLAUDE.md
```

`.gitignore` del repo de dotfiles (excluye lo pesado/efímero/sensible):

```

*
!projects/
!projects/**/memory/
!projects/**/memory/**
!commands/      !commands/**
!agents/        !agents/**
!hooks/         !hooks/**
!CLAUDE.md
# nunca:
settings*.json
**/cache/**
sessions/  history.jsonl  downloads/  plugins/  shell-snapshots/

```

```

gh repo create claude-config --private --source=. --remote=origin
git add -A && git commit -q -m "chore: dotfiles de Claude (memoria + comandos)"
git push -u origin main

```

7b. En la Mac nueva (traer la memoria)

```

cd ~/.claude
git clone https://github.com/rubatrejo/claude-config.git _claude-config
# copia/symlinkea el contenido dentro de ~/.claude conservando rutas:
rsync -a _claude-config/ ~/.claude/ # respeta projects/, commands/, agents/, hooks/, CLAUDE.md

```

Nota del hash: si el usuario de macOS difiere, la carpeta `projects/-Users-...-Kiosk-Portrait-Old/` tendrá otro nombre en la Mac nueva. Arranca `claude` una vez (crea la carpeta con el hash correcto), cierra, y mueve el contenido de `memory/` del hash viejo al nuevo.

7c. Alternativa cómoda (opcional)

Symlink de la carpeta `memory/` a **iCloud Drive** para sync automático sin comandos. Más cómodo, pero con riesgo de conflictos si editas en dos Macs casi a la vez. El repo git (7a/7b) es la opción robusta y recomendada.

8. Flujo de trabajo diario entre las dos Macs

```

# Al empezar:
git pull                                # repo del proyecto
( cd ~/.claude/_claude-config && git pull ) # memoria de Claude
claude → /iniciar                       # lee STATE.md, ROADMAP, verifica skills

# ... trabajar (plan mode, verificar, commit, push según CLAUDE.md) ...

# Al terminar:
/terminar                                # actualiza .planning/STATE.md y propone commit
# commit + push del proyecto (auto-deploy a Vercel)
( cd ~/.claude/_claude-config && git add -A && git commit -m "memoria: <fecha>" && git push )

```

Regla de oro: una Mac a la vez. Si saltas de máquina sin `push` / `pull`, git te hará resolver conflictos en `STATE.md` y en la memoria.

9. Notas heredadas (gotchas conocidos)

- **Login del Studio en `localhost` NO funciona.** La OAuth App de GitHub es de producción (`AUTH_TRUST_HOST=true`, sin `AUTH_URL` local) → el `redirect_uri` sale localhost y esa app no lo admite. En la Mac nueva, igual que en esta: se trabaja con el **preview local** o **desplegando a prod** (`trueomni-studio.vercel.app`). Si quieres login local, crea una OAuth App de **dev** con callback `http://localhost:3000`.
 - **Email de commits para Vercel.** Usa `rubat.rejo@gmail.com` como `git config user.email`; Vercel rechaza deploys cuyo author email no esté vinculado a la cuenta con acceso al proyecto (Hobby tier). Verifica con `git log -1 --format='%ae'`.
 - **`.env.local` nunca a git.** Está en `.gitignore`; si cambias un secret, vuelve a ponerlo en Vercel y haz `vercel env pull` en ambas Macs (single source of truth).
 - **`graphify-out/` no se versiona** (pesa ~9.5 MB); se regenera solo en cada commit vía el hook de Husky.
 - **Auto-deploy:** tras `push` a `main`, Vercel **espera** a que pase el GitHub Action "Production build" (~3-6 min) antes de crear el deploy. Cero deploys en ese lapso es normal; no re-dispares con commits vacíos.
-

Responsable: Rubén · **Repo proyecto:** `rubatrejo/TrueOmni-Studio` · **Repo memoria:** `rubatrejo/claude-config` (privado)